

CENTRO UNIVERSITÁRIO DE BRASÍLIA (CEUB)
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA DE PROJETO INTEGRADOR III

22300926 Raul Falluh Fragoso de Mendonça

22250549 Rodrigo Cesar da Silva Castro

22301814 Rodrigo Hawerroth Lemos

22253048 Rafael Irvine Sousa Senra

SISTEMA COISAS DE GARAGEM
DOCUMENTO DE ARQUITETURA DE SOFTWARE

1. INTRODUÇÃO

O projeto **Coisas de Garagem** foi concebido como uma plataforma digital voltada à organização e ao apoio de vendas de garagem, permitindo que produtos físicos sejam cadastrados, consultados e acompanhados por meio de recursos web. A proposta do sistema é aproximar o processo tradicional de venda presencial de uma experiência mais estruturada, em que vendedores podem registrar seus itens e compradores podem acessar informações detalhadas com maior praticidade, inclusive por meio de QR Code.

A arquitetura atual do projeto reflete a evolução do sistema ao longo das disciplinas de projeto integrador. A aplicação passou a adotar uma estrutura full stack baseada em separação clara entre interface, lógica de negócio e persistência de dados. Essa organização foi escolhida com o objetivo de facilitar a manutenção do código, melhorar a divisão de responsabilidades entre os integrantes da equipe e permitir que o sistema seja executado tanto em ambiente local de desenvolvimento quanto em cenário de implantação em nuvem.

Este documento apresenta a arquitetura atual do sistema, descrevendo seus componentes principais, a relação entre as camadas da aplicação, a estratégia de persistência e autenticação, bem como a distinção entre o ambiente utilizado durante o desenvolvimento e o ambiente previsto para produção.

2. OBJETIVO DA ARQUITETURA

A arquitetura do Coisas de Garagem foi definida para atender às necessidades funcionais e técnicas do projeto, oferecendo uma base estável para o desenvolvimento da aplicação e para sua evolução. A proposta é garantir que o sistema consiga suportar o fluxo principal de cadastro de produtos, autenticação de usuários, leitura de QR Code, registro de compras e acompanhamento de informações relevantes ao vendedor e ao comprador.

Ao mesmo tempo, a arquitetura busca manter o projeto suficientemente organizado para fins acadêmicos e práticos, permitindo que frontend, backend e banco de dados possam ser compreendidos e desenvolvidos de forma articulada, sem comprometer a coerência do sistema como um todo.

3. VISÃO GERAL DA SOLUÇÃO

A solução é composta por três núcleos centrais. O primeiro é a interface web, responsável pela interação direta com o usuário. O segundo é a API backend, onde se concentram as regras de negócio, as rotas protegidas, a autenticação e a comunicação com o banco de dados. O terceiro é a camada de persistência, responsável pelo armazenamento estruturado das informações da aplicação.

Na configuração atual, o frontend foi desenvolvido com **React**, utilizando **Vite** e **TypeScript**. O backend foi construído com **NestJS**, também em **TypeScript**, com organização modular. A persistência é feita em **PostgreSQL**, acessado por meio do **Prisma**. Em ambiente de produção, o banco é planejado para uso com **Neon**, enquanto em ambiente local de desenvolvimento a equipe pode utilizar PostgreSQL executado em contêineres Docker.

Essa separação entre camadas foi importante para a consolidação do projeto, pois permitiu à equipe maior controle sobre a lógica da aplicação e maior clareza na divisão técnica das responsabilidades.

4. CAMADA DE APRESENTAÇÃO

A camada de apresentação do sistema é composta por uma aplicação web desenvolvida em React, com apoio do Vite para empacotamento e execução em ambiente de desenvolvimento. A escolha dessa stack permitiu maior produtividade na construção da interface e facilitou a criação de componentes reutilizáveis, páginas modulares e rotas organizadas.

A interface contempla os principais fluxos do sistema, incluindo autenticação, cadastro e visualização de produtos, acesso a páginas públicas, área do vendedor, área do comprador, consulta de itens por QR Code e exibição de dados relacionados a compras e vendas. O frontend foi estruturado para funcionar de forma responsiva, o que é coerente com a proposta de uso em dispositivos móveis, especialmente no fluxo de leitura de QR Code por compradores.

O gerenciamento de rotas é realizado com React Router. O estado global da aplicação é controlado com Zustand, e os formulários utilizam React Hook Form com validação por Zod. A interface também utiliza bibliotecas auxiliares para gráficos, animações, geração de arquivos PDF e leitura de QR Code via navegador.

A comunicação com o backend é feita por chamadas HTTP centralizadas em uma camada de serviços. A URL base da API é configurada por variável de ambiente, permitindo alternância entre execução local e integração com a aplicação implantada.

5. CAMADA DE APLICAÇÃO E REGRAS DE NEGÓCIO

A camada de aplicação é implementada com NestJS e concentra a maior parte da lógica do sistema. Sua estrutura modular contribui para a organização do projeto e facilita tanto a manutenção quanto a escalabilidade futura. A API opera com rotas versionadas e possui documentação automática via Swagger, o que facilita testes, validação e entendimento técnico durante o desenvolvimento.

No estado atual do projeto, os módulos mais relevantes da aplicação envolvem autenticação, usuários, produtos, compras, métricas e QR Codes. O módulo de autenticação é responsável pelo registro e login de usuários, além da emissão e validação de tokens JWT. O módulo de usuários trata das informações básicas relacionadas às contas cadastradas. O módulo de produtos gerencia o cadastro, a edição, a exclusão e a disponibilidade dos itens anunciados. O módulo de compras registra as transações e organiza o histórico de aquisições e vendas. O módulo de métricas consolida dados relevantes para o painel do vendedor. Já o módulo de QR Code viabiliza a geração e o tratamento das informações associadas ao código utilizado para acessar rapidamente a página pública de um produto.

A lógica do backend foi estruturada para manter as regras mais importantes do domínio no servidor. Isso inclui validação de acesso, consistência de operações de compra, controle de disponibilidade do produto e associação correta entre comprador, vendedor e item negociado.

6. PERSISTÊNCIA E MODELO DE DADOS

A persistência do sistema é baseada em PostgreSQL, com acesso mediado pelo Prisma. Essa combinação oferece um equilíbrio interessante entre modelagem relacional, segurança de tipagem e facilidade de manutenção. O schema da aplicação contempla as entidades principais necessárias ao funcionamento da plataforma.

O modelo atual é centrado em usuários, produtos e compras. Usuários podem cadastrar itens para venda, consultar produtos, realizar compras e acessar funcionalidades específicas da plataforma. Cada produto possui vínculo com um vendedor, além de atributos como nome, descrição, preço, categoria, condição, imagem e identificador de QR Code. As compras registram a relação entre comprador, vendedor e produto, armazenando ainda informações complementares como método de pagamento e status da transação.

O uso do Prisma contribui para a consistência do acesso ao banco, especialmente em operações que exigem integridade entre múltiplas tabelas, como o registro de compras com atualização de disponibilidade do produto.

7. AUTENTICAÇÃO E CONTROLE DE ACESSO

A autenticação do sistema é baseada em JWT. Após login ou cadastro, o backend emite um token que é utilizado pelo frontend para acessar rotas protegidas. Esse mecanismo permite que o sistema mantenha sessões autenticadas de forma simples e adequada ao contexto da aplicação web.

As senhas são armazenadas de forma criptografada com bcrypt, e o backend utiliza validação estruturada dos dados recebidos nas requisições. Essa abordagem melhora a segurança e contribui para a robustez geral da aplicação.

A separação entre comprador e vendedor aparece principalmente na experiência de uso da interface e na organização dos fluxos do sistema. A documentação arquitetural, neste momento, não trata essa distinção como uma modelagem aprofundada de múltiplos perfis de autorização, já que essa formalização ainda não constitui o foco principal da entrega atual.

8. AMBIENTE DE DESENVOLVIMENTO

Durante o desenvolvimento do projeto, a equipe pode executar o sistema localmente, com frontend e backend rodando em ambiente de desenvolvimento e o banco de dados disponível por meio de contêiner Docker com PostgreSQL. Essa configuração facilita a padronização do ambiente entre os integrantes do grupo e reduz divergências de execução entre diferentes máquinas.

Nesse cenário, o frontend é iniciado localmente em modo de desenvolvimento, o backend é executado com NestJS em ambiente Node.js, e o banco é levantado via Docker Compose. Esse arranjo é útil para testes, validações de fluxo, ajustes de interface, revisão de rotas e acompanhamento da integração entre as camadas da aplicação.

A adoção de um ambiente local separado também é importante para o andamento do projeto na disciplina, pois permite desenvolvimento incremental sem depender constantemente do ambiente final de implantação.

9. AMBIENTE DE PRODUÇÃO

Para a arquitetura de produção, o projeto prevê uma separação mais próxima de um cenário real de uso. Nesse contexto, o frontend é planejado para implantação em plataforma adequada à hospedagem de aplicações web modernas, como a Vercel. O backend é destinado a serviço de execução Node.js, como o Render. O banco de dados, por sua vez, utiliza PostgreSQL gerenciado em nuvem por meio do Neon.

Essa organização permite desacoplamento entre as camadas da solução, além de facilitar a manutenção e a futura evolução do sistema. Também oferece uma estrutura mais adequada à apresentação e à validação do

projeto em ambiente remoto, o que é especialmente relevante para demonstrações, testes externos e consolidação da aplicação fora do computador dos desenvolvedores.

É importante destacar que, no documento, o ambiente de produção não substitui o ambiente local. Ambos coexistem com propósitos diferentes. O primeiro atende à implantação e disponibilização do sistema. O segundo atende ao desenvolvimento e à validação contínua da aplicação pela equipe.

10. FLUXOS PRINCIPAIS DO SISTEMA

Um dos fluxos centrais do sistema é o cadastro de produtos. O vendedor acessa a área correspondente no frontend, preenche os dados do item e envia essas informações à API. O backend valida os dados, gera o identificador necessário e registra o produto no banco. A partir disso, o item passa a compor a base do sistema e pode ser associado a QR Code para consulta rápida.

Outro fluxo importante é a consulta de produtos por QR Code. O comprador utiliza a câmera do navegador para ler o código disponibilizado fisicamente junto ao item. O frontend interpreta o conteúdo lido e consulta o backend, que retorna as informações correspondentes ao produto. Esse fluxo é um dos pontos que diferenciam o projeto, pois integra o ambiente físico da venda de garagem com a experiência digital da plataforma.

O fluxo de compra também ocupa papel central na arquitetura. Quando um comprador realiza uma operação de compra, o backend valida a disponibilidade do item e registra a transação no banco, atualizando o estado do produto. Essa operação é importante porque concentra uma das regras mais sensíveis do sistema: impedir inconsistências entre itens disponíveis e itens já negociados.

Além disso, o sistema possui um fluxo de acompanhamento gerencial por parte do vendedor, em que informações agregadas são consultadas pela área de analytics e exibidas na interface como apoio ao monitoramento da atividade comercial.

11. INTEGRAÇÕES E FUNCIONALIDADES EM EVOLUÇÃO

A arquitetura atual contempla funcionalidades já implementadas e outras ainda em evolução. Entre as que já estão consolidadas estão a autenticação básica, o cadastro e gerenciamento de produtos, a leitura de QR Code via navegador, o registro de compras e a estrutura principal da interface web.

Por outro lado, a integração com **AbacatePay/PIX** deve ser tratada, no momento, como funcionalidade em desenvolvimento. A documentação do projeto já indica essa direção, e ela faz sentido do ponto de vista do

domínio da aplicação, mas ainda deve ser apresentada como parte do processo evolutivo do sistema, e não como módulo totalmente consolidado na versão atual.

Esse cuidado é importante para manter a coerência entre o que será apresentado em sala e o que já está efetivamente implementado no projeto.

12. CONSIDERAÇÕES TÉCNICAS

A arquitetura atual representa um avanço importante na maturidade do projeto. A migração para uma estrutura com React, NestJS, Prisma e PostgreSQL trouxe mais controle sobre a aplicação, maior clareza organizacional e uma base mais adequada para continuidade do desenvolvimento.

Ao mesmo tempo, como se trata de um projeto acadêmico em andamento, ainda existem pontos de refinamento documental e técnico. Parte da documentação anterior foi produzida em momento distinto da evolução do sistema e, por isso, nem sempre reflete a arquitetura atual. A atualização proposta neste documento busca justamente consolidar uma visão técnica compatível com o estágio atual do projeto.

Essa consolidação é importante não apenas para a apresentação da disciplina, mas também para o alinhamento interno da equipe, uma vez que a documentação passa a servir como referência comum para entendimento da solução.

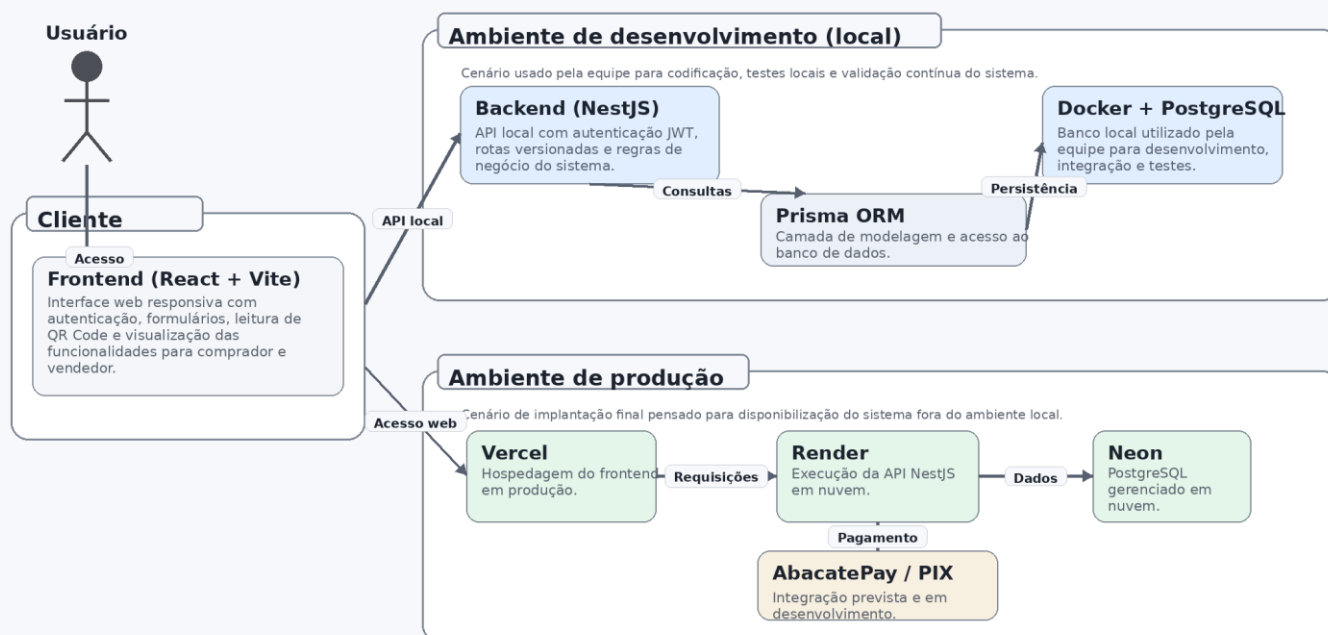
13. CONCLUSÃO

A arquitetura do **Coisas de Garagem** foi estruturada para sustentar uma aplicação web voltada à digitalização do processo de venda de garagem, conectando vendedores, compradores e produtos em uma plataforma organizada e extensível. A escolha por uma arquitetura em camadas, com frontend desacoplado do backend e persistência relacional, oferece uma base sólida para o escopo proposto na disciplina.

A separação entre ambiente de desenvolvimento e ambiente de produção reforça a consistência do projeto e demonstra preocupação com aspectos reais de construção de software. Em desenvolvimento, a equipe conta com um ambiente local controlado, apoiado por Docker e PostgreSQL. Em produção, a arquitetura prevê o uso de serviços em nuvem como Vercel, Render e Neon, adequados à disponibilização da solução.

No contexto da disciplina, essa arquitetura é suficiente para demonstrar domínio sobre tecnologias atuais, organização técnica do projeto e coerência entre requisitos, implementação e documentação. A partir dela, a equipe pode seguir com a atualização dos demais artefatos, especialmente README, documento de implantação e apresentação final, mantendo uma narrativa única e tecnicamente consistente.

Arquitetura de Alto Nível – Coisas de Garagem



O diagrama distingue dois cenários complementares: no desenvolvimento, a equipe trabalha localmente com NestJS, Prisma e PostgreSQL em Docker; na produção, a arquitetura final prevê frontend em Vercel, backend em Render e banco no Neon.

A arquitetura do sistema foi organizada em dois contextos complementares: o ambiente de desenvolvimento e o ambiente de produção. No lado do cliente, o usuário acessa a plataforma por meio do navegador, utilizando o frontend desenvolvido em React com Vite. É nessa camada que ficam concentradas a interface web, a autenticação visual, os formulários, a leitura de QR Code e a navegação entre as funcionalidades destinadas a compradores e vendedores.

No ambiente de desenvolvimento, o frontend se comunica com uma API backend construída em NestJS, executada localmente pela equipe. A persistência dos dados, nesse cenário, é feita com PostgreSQL em contêiner Docker, utilizando o Prisma como camada de acesso e modelagem dos dados. Essa configuração foi adotada para facilitar testes, padronizar o ambiente entre os integrantes do grupo e permitir validações contínuas durante a construção do projeto.

No ambiente de produção, a arquitetura prevista separa as responsabilidades de forma mais próxima de um cenário real de uso. O frontend é disponibilizado em uma plataforma de hospedagem web, representada no diagrama pela Vercel. O backend é implantado em um serviço de execução em nuvem, representado pelo Render. Já o banco de dados passa a utilizar PostgreSQL gerenciado no Neon. Além disso, a integração com AbacatePay/PIX aparece no diagrama como funcionalidade em desenvolvimento, indicando uma evolução planejada do sistema, mas ainda não tratada como módulo totalmente consolidado.

Essa separação entre desenvolvimento e produção permite documentar com clareza tanto a forma como a equipe trabalha localmente quanto a arquitetura final pensada para a disponibilização do sistema.